

Récapitulatif du 9 avril 2026

- [1] Singh, R., Ren, J., & Lin, X. (2023). A review of deep reinforcement learning algorithms for mobile robot path planning. *Vehicles*, 5(4), 1423-1451.
- [2] Reijnen, R., Zhang, Y., Nuijten, W. P. M., Senaras, C., & Goldak, M. (2020). Combining Deep Reinforcement Learning with Search Heuristics for Solving Multi-Agent Path Finding in Segment-based Layouts. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, Canberra, ACT, Australia, 2020, pp. 2647-2654, doi: 10.1109/SSCI47803.2020.9308584.
- [3] Wang, B., Liu, Z., Li, Q., & Prorok, A. (2020). Mobile robot path planning in dynamic environments through globally guided reinforcement learning. In *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6932-6939, Oct. 2020, doi: 10.1109/LRA.2020.3026638.
- [4] Alexander Amini. MIT 6.S191: Reinforcement Learning [vidéo en ligne]. Youtube, 31 mars 2025 [vue le 1er avril 2026]. Disponibilité et accès <https://www.youtube.com/watch?v=to-IHJfK4pw>.

Algorithmes de Deep RL pour le robot path planning

Type	Exemples	Description	Commentaires
Value-based	DQN [1, 4], DDQN [1], D3QN [1]	Approxime la fonction $Q(s, a)$ en utilisant un réseau de neurones profonds.	Simple, mais limité aux actions discrètes
Policy-based	Policy gradient [1]	Apprend la meilleure politique π en ajustant les probabilités d'action à partir des récompenses.	Adapté aux actions continues, mais instable
Actor-critic	DDPG [1, 4], A3C [1], PPO [1], SAC [1]	Combine un actor qui génère chaque action et un critic qui l'évalue (selon Q ou V).	Très utilisé dans les approches modernes, mais assez complexe
Hybrides	WHCA*-RL [2], G2RL [3]	Combine du RL avec une méthode plus classique (déterministe ou heuristique).	Développé pour le cas multi-agent, mais assez complexe

Value-based methods

Description : Approxime la fonction $Q(s, a)$ en utilisant un réseau de neurones profonds.

Méthode	Explication	Remarques
DQN	L'agent choisit ses actions avec une politique ϵ-gloutonne, donc la fonction valeur est apprise indépendamment de la politique utilisée pour agir. Deux réseaux de neurones sont utilisés : 1. Un prediction network qui prend en entrée s et prédit $Q_{pred}(s, a)$, mis à jour en continu. 2. Un target network qui calcule la cible y, mis à jour périodiquement. $y = r + \gamma \max_{a'} Q_{target}(s', a')$ Pour cela, DQN garde les transitions en mémoire dans le replay buffer (s, a, r, s'). Le modèle minimise l'erreur quadratique entre la cible et la valeur prédite avec une descente de gradient (MSE).	Cette méthode est assez simple, mais les valeurs de la fonction Q peuvent être facilement surestimées, car on utilise le même réseau pour choisir l'action a' et évaluer sa valeur.
DDQN	On sépare la sélection de l'action optimale et son évaluation entre les deux réseaux de neurones. Le prediction network choisit la meilleure action a^* et le target network va évaluer sa valeur et calculer la cible y. $a^* = \arg \max_a Q_{pred}(s', a), \quad y = r + \gamma Q_{target}(s', a^*)$	Ici, on a réduit la surestimation des valeurs de Q.
D3QN	L'algorithme décompose la valeur de Q en une valeur-état V et une valeur-avantage A, qui mesure à quel point une action a est avantageuse dans l'état s. On sépare le réseau en deux streams : l'une qui estime la valeur de l'état $V(s)$, et l'autre qui estime l'avantage de chaque action $A(s, a)$, puis on les combine (le plus souvent, en soustrayant la moyenne). $Q(s, a) = V(s) + A(s, a) - \frac{1}{ A } \sum_{a'} A(s, a')$	Le modèle apprend quels états sont plus avantageux que d'autres, sans devoir calculer l'effet de chaque action sur l'état. ça permet également de réduire la variance dans l'estimation de Q.

Policy-based methods

Description : Apprend la meilleure politique π en ajustant les probabilités d'action à partir des récompenses.

Méthode	Explication	Remarques
Policy gradient	L'objectif est d'apprendre une politique optimale $\pi_\theta(a s)$, sans passer par la fonction Q. Pour cela, on maximise l'espérance de la récompense cumulée $J = E_{\pi_\theta} [G_0]$, où $G_t = \sum_{k=0}^{+\infty} \gamma^k r_{t+k}$. On peut réécrire $J = E_{\pi_\theta} [Q(s, a)] = \sum_a \pi(a s) Q(s, a)$ et le gradient de cette fonction est défini de cette façon : $\nabla_\theta J(\theta) = E_\pi [\nabla_\theta \log \pi_\theta(a s) Q(s, a)]$. Durant l'entraînement, on exécute la politique π_θ et on stocke les transitions (s, a, r) jusqu'à la fin de l'épisode. On en déduit le gain G_t pour chaque étape et on met à jour les paramètres via une descente de gradient stochastique : $\theta' = \theta + \alpha \log \nabla_\theta \pi_\theta(a s) \times G_t$, où α est le learning-step.	Intuitivement, on augmente la probabilité des actions qui ont reçu une forte récompense et on diminue la probabilité de celles qui ont reçu une faible récompense. Cependant, le gradient peut présenter une grande variance, surtout si la récompense cumulée varie beaucoup. Cela peut rendre l'apprentissage instable.

Actor-critic methods

Description : Combine un actor qui génère chaque action et un critic qui l'évalue (selon Q ou V).

Méthode	Explication	Remarques
DDPG	L'actor prend l'état s et génère une action continue a en estimant $\pi(s)$. Le critic prend l'état s et l'action choisie a, et évalue cette action en estimant $Q(s, a)$. Pour stabiliser l'apprentissage, les deux réseaux disposent de copies (target actor et target critic) qui	Cette méthode fonctionne assez bien pour des espaces d'actions continues, mais elle est

	sont mises à jour progressivement (soft update) : $\theta'_{target} = (1 - \tau)\theta_{target} + \tau\theta_{main} \text{ où } \tau \text{ est très petit.}$ Les transitions (s, a, r, s') sont stockées dans le replay buffer. Comme on est dans un cas continu, on approxime $\max_{a'} Q(s, a) \approx \max_{\pi} Q(s, \pi(s))$. Le critic est optimisé avec une MSE entre la valeur estimée $Q(s, a)$ et la cible : $y = r + \gamma Q_{critic\ target}(s', \pi_{actor\ target}(s'))$ L'actor est mis à jour en maximisant Q via policy gradient, en utilisant les réseaux principaux : $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \pi(s) \times \nabla_a Q(s, a)$	assez sensible aux hyperparamètres et au risque de surestimation du critic. Elle peut toutefois être étendue au cadre multi-agents : le critic peut recevoir des informations d'autres agents pour mieux évaluer les actions, et l'actor reste centré sur ses propres informations pour décider de ses actions.
A3C	Plusieurs threads actor-learner interagissent de manière parallèle avec un environnement afin d'apprendre simultanément la politique et la fonction de valeur. Puis, ils mettent à jour le réseau global de manière périodique, ce qui accélère l'apprentissage. Comme pour D3QN, cet algorithme utilise une fonction avantage pour réduire le gradient de la variance. Contrairement à DDPG c'est une méthode on-policy, il n'y pas de replay buffer. Les paramètres du réseau sont mis à jour comme suit : $\theta' = \theta + \alpha \log \nabla_{\theta} \pi_{\theta}(a s) \times A(s, a) + \beta \nabla_{\theta} H(\pi_{\theta}(s))$ où H est une entropie et α, β sont des paramètres du réseau.	Les traitements peuvent être réalisés de manière parallèle, ce qui rend l'apprentissage plus rapide. Cependant, elle est principalement utilisée dans des espaces d'action discrets.
PPO	C'est une méthode de RL on-policy basée sur l'algorithme TRPO (trust region policy optimization). TRPO ajoute une contrainte sur la divergence de Kullback-Leibler entre l'ancienne et la nouvelle politique, pour éviter des changements trop brutaux. Plus particulièrement, en appliquant le théorème de policy gradient, on a : $\nabla_{\theta} J(\theta) = E_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a s) Q(s, a)] \text{ ou } \nabla_{\theta} J(\theta) = E_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a s) A(s, a)]$ Qui peut être obtenu en différenciant : $J(\theta) = E_{\pi} [\log \pi_{\theta}(a s) A(s, a)] \text{ ou } J(\theta) = E_{\pi} \left[\frac{\pi_{\theta}(a s)}{\pi_{old}(a s)} A(s, a) \right]$ Donc TRPO utilise : $J(\theta) = E_{\pi} \left[\frac{\pi_{\theta}(a s)}{\pi_{old}(a s)} A(s, a) - \beta D_{KL}(\pi_{\theta, old} \pi_{\theta}) \right]$ pour pénaliser l'écart entre les politiques. PPO simplifie cette approche en utilisant un clipping du ratio des probabilités entre les deux politiques,	Cette méthode peut être appliquée à des espaces d'action discrets ou continus. Cependant, elle est assez sensible aux paramètres.

	en le forçant à être entre $1 - \epsilon$ et $1 + \epsilon$. Soit : $J(\theta) = E_{\pi_{\theta}} \left[\min \left(\frac{\pi_{\theta}(a s)}{\pi_{old}(a s)} A(s, a), \text{clipped} \left(\frac{\pi_{\theta}(a s)}{\pi_{old}(a s)} \right) \times A(s, a) \right] \right]$	
SAC	En plus de maximiser la récompense cumulée, cette méthode utilise une fonction objective qui permet également de maximiser l'entropie de la politique, afin de favoriser l'exploration et d'éviter les optimums locaux. En utilisant une politique stochastique et un replay buffer, elle améliore l'efficacité de l'apprentissage. Elle utilise également deux réseaux critics (double Q-learning) pour éviter la surestimation de la fonction Q.	Cette méthode offre une meilleure stabilité et apporte un bon compromis entre exploration et exploitation, ce qui évite une politique trop déterministe.

Hybrids methods

Description : Combine du RL avec une méthode plus classique (déterministe ou heuristique).

Méthode	Explication	Remarques
WHCA*-RL	WHCA*S est une adaptation du WHCA*, utilisée pour résoudre le problème de MAPF sur des grilles, au cas segmented-based : chaque agent est contraint de rester sur un segment prédéfini et d'occuper des segments aux alentours, au lieu d'occuper une case de la grille. Pour chaque agent, l'objectif du WHCA*S est de trouver un chemin optimal pour une fenêtre de taille W de manière séquentielle, et d'en exécuter K étapes parallèlement ($K < W$). Les conflits sont gérés en réservant les segments (une zone limite est déterminée autour des agents), et les embouteillages ne sont pas anticipés. Cependant, ces chemins partiels optimaux sont obtenus à l'aide d'algorithmes classiques tels que A^*, qui repose sur le chemin le plus court, ce qui n'est pas toujours le meilleur. WHCA*-RL utilise la méthode PPO pour décider localement de l'action. Pour cela, il définit un processus de décision semi-markovien pour apprendre la politique, qui correspond au tuple $\langle S, A, P, F, R \rangle$ avec S l'espace des état, A l'espace des actions, P la probabilité de transition, F la	Ce modèle est particulièrement adapté au multi-agent et a déjà été appliqué dans un contexte similaire au nôtre (robots porteurs de bagage). Il nécessiterait toutefois une adaptation de la modélisation du circuit. Cependant, il repose sur une fonction de récompense

	distribution du temps entre deux actions et R la fonction de récompense. Un état s est défini par les directions possibles, la distance vers la cible, et le nombre de pas disponibles sur le segment. Une action a correspond au choix d'une direction : les directions possibles sont toutes les mêmes, quel que soit l'état, certaines sont cependant invalides. 1. On initialise des scénarios. Pour chaque scénario, on initialise les chemins de certains AGV en utilisant WHCA*. 2. On choisit un AGV qui n'a pas de chemin prédéfini : c'est le learner agent. On prédit son action avec la méthode PPO. 3. Si l'action est valide, on le récompense et l'action est stockée dans le système de réservation du WHCA*S. Sinon, on le pénalise et l'agent doit choisir une autre action. 4. On répète jusqu'à avoir W actions valides, puis on met à jour la politique PPO avec les états et récompenses collectées. Donc : WHCA*S permet de planifier le chemin global et d'éviter les collisions et le RL permet de gérer la prise de décision localement pour éviter les embouteillages.	locale, qui ne prend pas vraiment en compte l'atteinte de l'objectif (dépôt, ramassage) mais uniquement la progression vers celui-ci. Il peut donc avoir des difficultés pour apprendre des stratégies globales optimales.
G2RL	Cette méthode résout le problème de path planning en environnement dynamique : l'objectif est d'atteindre une destination, tout en évitant des collisions avec des objets statiques ou dynamiques (robots). Elle repose sur un algorithme de path-planning hiérarchique qui combine une global guidance et un local RL planner. L'environnement est modélisé comme une grille, et l'idée consiste à optimiser le nombre de pas vers la destination tout en corrigeant le chemin global. Le système est composé de 4 modules : • Inputs : L'observation locale de l'agent est transformée en une image à 3 canaux (RGB) puis combinée à un canal qui correspond à la global guidance pour former l'input actuel. L'input final correspond à un historique des inputs. • Traitement de l'espace : On extrait les caractéristiques de l'input avec un CNN 3D, puis on les reshape en une séquence de vecteur 1D de la taille de l'historique. • Traitement du temps : On extrait les caractéristiques temporelles avec une couche LSTM, en agrégeant ensuite les vecteurs. • Politique : On estime la qualité de chaque pair état-action avec des couches FC, puis on choisit la meilleure. La fonction de récompense est telle que l'agent reçoit une petite pénalité s'il s'éloigne de la global guidance, une forte pénalité s'il entre en collision avec un objet, mais une récompense	Tout comme WHCA*-RL, le modèle est particulièrement adapté au multi-agent et peut être facilement généralisé à grande échelle. Il nécessiterait également une adaptation de la modélisation du circuit, sous forme de grille cette fois-ci. Encore une fois, il ne planifie pas globalement et donc sa qualité dépend de la guidance initiale utilisée

	proportionnelle au nombre de cases de la guidance rattrapées après déviation. Cette global guidance est calculée une seule fois au début de l'épisode, avec un algorithme classique telle que A* ou Dijkstra. Donc : G2RL apprend à localement corriger un chemin global, sans planification globale complète. Chaque agent considère une observation partielle et n'a donc pas besoin de connaître les trajectoires des autres. Ainsi, on peut entraîner le modèle sur un agent, et l'appliquer à plusieurs.	
--	---	--

Conclusion :

Objectif du projet : minimiser le temps nécessaire au ramassage et au dépôt de bagages par les AGV, tout en gérant les collisions.

Une première modélisation du problème repose sur une architecture non supervisée : Un agent par AGV, qui décide de l'action à partir de l'observation partielle de son environnement.

- Chaque AGV dispose d'une connaissance a priori de la carte, ainsi que de sa position dans celle-ci. Les obstacles dynamiques ne sont perçus que localement via l'observation partielle.
- L'observation partielle de l'environnement est recueillie à partir d'une caméra sur l'AGV, c'est son champ de vision.
- La caméra fournit une image qui sera encodée sous forme de grille, où chaque cellule correspond à un élément de l'environnement : l'AGV (au centre), le chemin, les obstacles fixes et les obstacles dynamiques. Cette grille correspond à une portion de la carte globale.
- Il faudra définir la fréquence de mise à jour de cette carte locale, qui dépend de la vitesse de l'agent et de l'unité de temps choisie. Dans un premier temps, on considérera que la carte est consultée après un déplacement de t pixels.

Le vecteur d'états comprend : la position de l'AGV dans la carte globale, la carte locale encodée, ainsi que l'information indiquant s'il transporte ou non un bagage

Le vecteur d'actions est composé de la direction de déplacement et du différentiel de vitesse.

Deux idées de système de récompense sont envisagées :

- Idée 1 : forte pénalité en cas collision, faible pénalité en cas décélération, et récompense en cas d'accélération, en prenant en compte l'atteinte de la vitesse maximale (risque de difficulté durant l'apprentissage, ex: il faut toujours accélérer).
- Idée 2 : forte pénalité en cas de collision, forte récompense lorsqu'un bagage est récupéré ou déposé, faible pénalisé sinon.

Objectif pour le moment : Étudier des modèles adaptés au cadre défini et aux hypothèses formulées, en tenant compte du critère de temps d'apprentissage, tout en réfléchissant à la mise en œuvre dans le simulateur.